

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	SHAFFRIA.A	Reg. No.:	192571047
Section / Batch:	B TECH CSBS	Date:	23.07.2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONCEPT MAPPING

Code of Conduct

I SHAFFRIA.A(192571047) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						

Overall Total (100)	
---------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q75. Consider the concept map: Recursion → Recursion Tree → Level Cost → Total Cost → Time Complexity. Explain how a recursive algorithm can be visualized using a recursion tree and how costs are distributed across levels. Extend the map by including Tree Height and Branching Factor as influencing nodes. Justify how summing level costs leads to final complexity. Interpret how this map simplifies analysis of recursive algorithms.

Q76. Consider the concept map: Algorithm → Loop Structure → Iterations → Operation Count → Time Complexity. Explain how iterative constructs contribute to operation count and complexity. Extend the map by including Single Loop and Nested Loop as branches. Justify how the number of iterations directly determines growth rate. Interpret how this mapping helps in analyzing iterative algorithms effectively.

RECURSIVE ALGORITHM ALGORITHM.

(15).

ALGORITHM : RECURSION.

Divide & conquer.

visualization : Recursion tree.

Tree structure.

Root node
(Level=0.)

Initial problem
cost.

Internal nodes
(Levels 1 to $b-1$)

subproblem division.
Work at each
level.

Leaf nodes
(Level $1 = h$)

Base
cases.
constant
cost.

Influencing parameters.

Branching factor (b)

Children per
node.

Tree height (h).

No. of levels
Determined by n and
 b (eg. $h = \log_b n$.)

calculating complexity

Σ summation of level
costs.

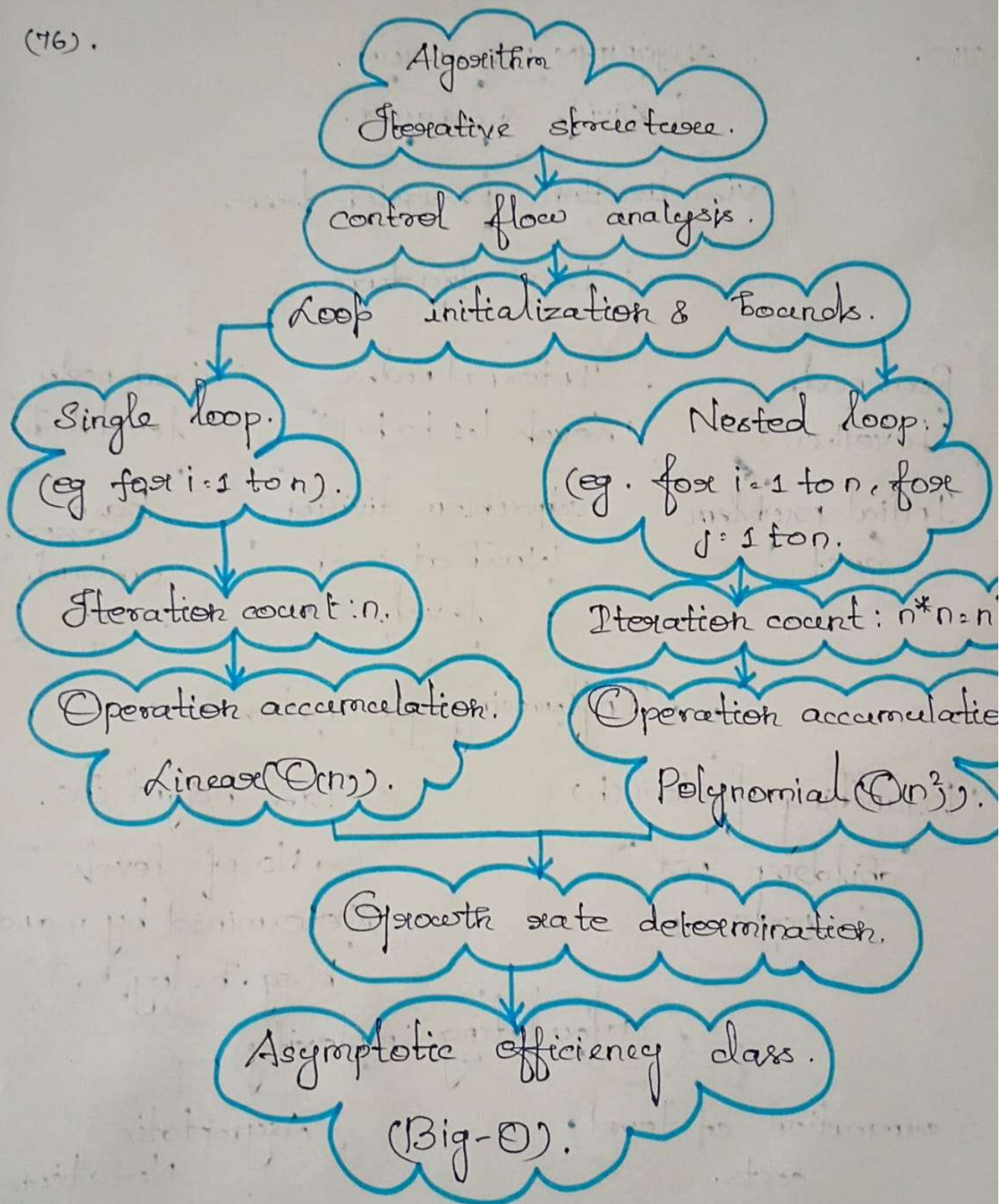
Total cost : Σ (level
costs)

Asymptotic
Notations

(Big-O).

Final time
complexity
(eg. $O(n \log n)$)

(76).



(((ITERATIVE ALGORITHM ANALYSIS)))

Question 75 Concept Map: Algorithm → Recursion → Recursion Tree → Level Cost → Total Cost → Time Complexity.

1. Visualizing via Recursion Tree & Cost Distribution:

When a recursive algorithm executes (such as a Divide and Conquer algorithm like Merge Sort), it breaks a problem of size n into smaller subproblems. A recursion tree is a graphical representation of this process where:

- Each node represents the cost of a subproblem processed at a given recursive stage.
- The execution is broken down into tiers or levels, where the cost at any single level is the sum of all subproblem costs active at that depth.

2. Extended Map: Branching Factor (b) and Tree Height (h):

To accurately analyze the tree, we incorporate two core structural parameters:

- Branching Factor (b): The number of children each parent node produces (i.e., how many independent subproblems a single problem splits into, such as $b = 2$ in standard Merge Sort).
- Tree Height (h): The maximum number of recursive steps required to reach the base case. The height is

fundamentally constrained by the input size n and branching factor, typically scaling as $h = \log_b n$.

3. Summation of Level Costs to Determine Total Cost:

The final execution time (Total Cost) of the recursive algorithm is derived by calculating the cumulative sum of all level costs from the root node ($l = 0$) down to the leaf nodes ($l = h$):

$$\text{Total Cost} = \sum_{l=0}^h (\text{Cost at level } l)$$

- If work is distributed evenly across all levels (e.g., $O(n)$ work per level across $\log n$ levels), the total complexity evaluates to $O(n \log n)$.

4. Interpretive Significance:

This mapping framework simplifies algorithm analysis by translating complex, non-linear recurrence relations into standard geometric or arithmetic series. It allows developers and analysts to isolate where computational bottlenecks occur—whether at the root, uniformly across levels, or concentrated at the leaf base cases.

Question 76 Concept Map: Algorithm → Loop Structure → Iterations → Operation Count → Time Complexity.

1. Iterative Constructs & Operation Accumulation:

Iterative algorithms rely on explicit looping constructs (such as for and while) to execute code sequences repeatedly. Every execution of an inner statement contributes to the cumulative operation count. As input data scales, the total frequency of these operations dictates the algorithm's running time efficiency.

2. Extended Branches: Single Loop vs. Nested Loops:

The structural complexity of iterative code branches into distinct patterns:

- **Single Loop (Linear Scaling):** A single loop iterating from 1 to n executes its inner statements exactly n times. This direct linear mapping yields an operation growth rate of $O(n)$.
- **Nested Loops (Polynomial Scaling):** A loop placed inside another loop (e.g., an outer loop running n times, and for every iteration, an inner loop running n times) multiplies the iteration space. The total operations scale to $n \times n = n^2$, resulting in a polynomial time complexity of $O(n^2)$.

3. Iterations Determining Growth Rate:

The mathematical relationship between input size n and the loop control bounds directly determines the asymptotic growth rate. If loop increments scale additively, growth is linear; if loop iterations multiply across dimensions (such as triangular iterations or multi-dimensional matrices), growth scales polynomially or exponentially.

4. Interpretive Significance:

This mapping provides a direct, predictable pathway for evaluating iterative code. By isolating loop structures, tracking control variable bounds, and converting them into mathematical summations, analysts can accurately forecast performance bottlenecks and check code scalability prior to deployment on large-scale datasets.
